

2.3.4 The Efficiency Evaluation Framework

کارکردگی کے تین سوال: کیا الگورتھم بڑے n پر چل سکتا ہے (Scale)؟

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 09 of 16

2.3.4 The Efficiency Evaluation Framework

LEARN IT

To evaluate an algorithm at a conceptual level, ask three questions:

1. **Does it scale?** If n grows 10 times, does time jump 10 times (linear) or 100 times (quadratic)? Quadratic scaling fails for large datasets.
2. **What is the bottleneck?** Focus on the most time-consuming section (usually the deepest loop) and try to improve that part.
3. **Is the cost worth it?** A theoretically faster $O(\log n)$ algorithm can be too complex to code and maintain. For a list of just 50 items, a simple $O(n)$ linear search is practically better because it is simple.

Does it scale?



Where is the bottleneck?



Is the extra complexity worth it?

WORKED EXAMPLE

Worked example: 50 names versus 1 million names.

Situation	Scale	Bottleneck	Worth the cost?	Choice
50 unsorted class names	Tiny n	A single linear scan	Binary search needs a sort first, which is extra work and extra code	Linear search $O(n)$
1,000,000 sorted account numbers, searched all day	$n \times 10$ compared with 100,000 is already huge; quadratic would be impossible	The search loop	Yes — binary search is standard and well understood	Binary search $O(\log n)$

REMEMBER IT

Framework mnemonic: **S-B-W** — **S**cale, **B**ottleneck, **W**orth it.

COMMON MISTAKE

Mistake: always choosing the theoretically fastest Big O.

Fix: question 3 exists for a reason. For $n = 50$, simple $O(n)$ beats a fragile $O(\log n)$ implementation.

اردو میں سمجھیں

کارکردگی کے تین سوال: کیا الگورتھم بڑے n پر چل سکتا ہے (Scale)؟ سب سے سست جگہ کون سی ہے (Bottleneck، عموماً گہرا لوپ)؟ کیا تیز تر طریقے کی پیچیدگی اس چھوٹے فائدے کے لائق ہے؟ پچاس ناموں کے لیے سادہ $O(n)$ تلاش عملی طور پر بہتر ہو سکتی ہے۔

CHECK YOURSELF

1. If n grows $10\times$, how much slower is a quadratic algorithm?
2. Where do you usually look for the bottleneck?
3. Why might linear search beat binary search for 50 items?

Answers: 1. About $100\times$ slower (10^2). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for $n = 50$ the simplicity of $O(n)$ is practically better.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The framework looks at:

- a. Steps, conditions, repetitions (then Big O)
- b. Only logos
- c. Only careers
- d. Only Figma

MCQ 2. "Number of steps" is:

- a. Basic operations in one pass / closed form
- b. The page count of the PDF
- c. OSI layers
- d. Sprint days

MCQ 3. You combine the three to:

- a. Justify a Big O class
- b. Pick a colour
- c. Pick a font
- d. Pick a career

MCQ 4. A 5-mark compare question wants:

- a. The framework on both algorithms + a verdict
- b. Only names
- c. Only Python libraries
- d. Only UX

MCQ 5. Constants matter at tiny n but the framework emphasises:

- a. Growth
- b. Hex colours
- c. Teacher names
- d. PDF margins

MCQ 6. If repetitions are nested $n \times n$, Big O is likely:

- a. n^2 regardless of a few extra ifs
- b. $\log n$
- c. 1
- d. $n!$

Extra short questions

1. State the efficiency evaluation framework.
2. Apply in one line to binary search.
3. Apply to bubble sort.
4. When would you still pick the slower class?

Extra long question

1. Using the framework, compare selection sort vs binary search as if a student mixed them up. Then say why they are not alternatives.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Count significant steps, conditions, repetitions; combine into time/space class; compare to requirements.
2. Few steps per shot, one/two conditions, $\log n$ repetitions $\rightarrow O(\log n)$ time, $O(1)$ space.
3. Compare+swap steps, a condition, nested repetitions $\rightarrow O(n^2)$.
4. Tiny n , need stability/simplicity, or the fast one needs a costly precondition (sort first).

Long

1. Different problems (order vs find). Framework still: both have loops; sort n^2 vs search $\log n$. Verdict: sort then many binary queries if the list is reused.